Practical-1 : studentclass.cpp

Design and implement a C++ program to create a class Student with data members for roll number, name, and marks. Implement member functions to accept and display the student details

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
   int rollno;
   string name;
   float marks;

public:
   void accept()
   {
      cout << "enter roll no";
      cin >> rollno;
      cout << "enter name";
      cin >> name;
      cout << "enter marks";
      cin >> marks;
   }
   void display()
   {
      cout << "roll no of student is" << rollno << endl;
      cout << "name of student is" << name << endl;
      cout << "marks of student" << marks << endl;
   }
};
int main()
{
   Student s;
   s.accept();
   s.display();
   return 0;
}
```

Practical 2: bankaccount.cpp (Constructor)

Design a C++ program to implement a class Bank Account that uses constructors to initialize account details and a destructor to indicate object destruction. Use a static data member to maintain the count of total bank accounts created.

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
private:
   int account_no;
   string name;
   float balance;
   static int count;

public:
   BankAccount(int acc, string nm, float bl)
   {
      account_no = acc;
      name = nm;
      balance = bl;
      count++;

      cout << "\nBank account created successfully!\n";
   }

   void display()
   {
      cout << "\n-----------------------------\n";
      cout << "Account Number : " << account_no << endl;
      cout << "Account Holder : " << name << endl;
      cout << "Account Balance: Rs. " << balance << endl;
      cout << "-----------------------------\n";
   }

   static void display_count()
   {
      cout << "\nTotal Bank Accounts: " << count << endl;
   }

   ~BankAccount()
   {
      cout << "Account " << account_no << " destroyed.\n";
```

```
    }
};

int BankAccount::count = 0;

int main()
{
    BankAccount b1(1, "Shrawani", 10000000);
    BankAccount b2(2, "Shravani", 5000000);

    cout << "\n===== BANK ACCOUNT DETAILS =====\n";

    b1.display();
    b2.display();

    BankAccount::display_count();

    cout << "\nProgram ended.\n";

    return 0;
}
```

Practical 3 : employeinheritance.cpp (inheritance)

Develop a C++ program demonstrating inheritance by creating a base class Employee and derived classes such as Permanent Employee and Contract Employee. Override member functions in derived classes to calculate and display employee salary details.

```
 #include <iostream>
#include <string>
using namespace std;

class Employee
{
protected:
   int empId;
   string empName;

public:
   Employee(int id, string name)
   {
      empId = id;
      empName = name;
   }

   virtual void calSalary()
   {
      cout << "Salary is not defined.\n";
   }

   void display()
   {
      cout << "Employee ID   : " << empId << endl;
      cout << "Employee Name : " << empName << endl;
   }

   virtual ~Employee() {}
};

class PermanentEmp : public Employee
{
private:
   double basicSal;
   double allowance;

public:
   PermanentEmp(int id, string name, double basic, double allow)
      : Employee(id, name)
```

```
    {
        basicSal = basic;
        allowance = allow;
    }

    void calSalary() override
    {
        cout << "Salary       : Rs. "
             << basicSal + allowance << endl;
    }
};

class ContractEmp : public Employee
{
private:
    int hours;
    double rate;

public:
    ContractEmp(int id, string name, int h, double r)
        : Employee(id, name)
    {
        hours = h;
        rate = r;
    }

    void calSalary() override
    {
        cout << "Salary       : Rs. "
             << hours * rate << endl;
    }
};

int main()
{
    PermanentEmp p(102, "Shrawani", 1000000, 500000);
    ContractEmp c(190, "Shravani", 3, 4000);

    cout << "===== PERMANENT EMPLOYEE =====\n";
    p.display();
    p.calSalary();

    cout << "\n===== CONTRACT EMPLOYEE =====\n";
    c.display();
    c.calSalary();
    return 0;
}
```

Practical 4 : Area.cpp (polymorphism) :

Write a C++ program to demonstrate runtime polymorphism by creating a base class Shape with a virtual function to calculate area. Implement derived classes like Rectangle and Circle that override the base class function.

```
# include <iostream>
using namespace std;

// Base class
class Shape {
public:
   // Virtual function
   virtual void calculateArea() {
      cout << "Area of Shape" << endl;
   }
};

// Derived class Rectangle
class Rectangle : public Shape {
private:
   float length, width;

public:
   Rectangle(float l, float w) {
      length = l;
      width = w;
   }

   // Overriding virtual function
   void calculateArea() override {
      float area = length * width;
      cout << "Area of Rectangle = " << area << endl;
   }
};

// Derived class Circle
class Circle : public Shape {
private:
   float radius;

public:
   Circle(float r) {
      radius = r;
   }
```

```
    // Overriding virtual function
    void calculateArea() override {
        float area = 3.14159 * radius * radius;
        cout << "Area of Circle = " << area << endl;
    }
};

int main() {
    // Base class pointer
    Shape* shape;

    Rectangle rectangle(10, 5);
    Circle circle(7);

    // Runtime polymorphism
    shape = &rectangle;
    shape->calculateArea();

    shape = &circle;
    shape->calculateArea();

    return 0;
}
```

Practical 5 : try.cpp (exception handling)

Design a C++ program to perform arithmetic division of two numbers. Handle exceptions such as division by zero and invalid input using exception handling mechanisms.

```
#include <iostream>
using namespace std;

int main() {
   double num1, num2;

   try {
      cout << "Enter the first number: ";
      if (!(cin >> num1)) {
         throw "Invalid input! Please enter a number.";
      }

      cout << "Enter the second number: ";
      if (!(cin >> num2)) {
         throw "Invalid input! Please enter a number.";
      }

      // Check for division by zero
      if (num2 == 0) {
         throw "Division by zero is not allowed!";
      }

      double result = num1 / num2;

      cout << "Result = " << result << endl;
   }
   catch (const char* message) {
      cout << "Exception: " << message << endl;
   }

   return 0;
}
```

Practical 6 : Emp.cpp (file handling)

Create a C++ program to store and retrieve records of employees using file handling. Implement a class Employee and perform write and read operations on files using C++ file streams.

```
#include <iostream>
#include <fstream>
using namespace std;

class Employee {
private:
   int empId;
   char name[50];
   float salary;

public:
   // Function to accept employee details
   void getData() {
      cout << "Enter Employee ID: ";
      cin >> empId;

      cout << "Enter Employee Name: ";
      cin.ignore();
      cin.getline(name, 50);

      cout << "Enter Salary: ";
      cin >> salary;
   }

   // Function to display employee details
   void showData() {
      cout << "\nEmployee ID: " << empId;
      cout << "\nEmployee Name: " << name;
      cout << "\nSalary: " << salary << endl;
   }
};

int main() {
   Employee emp;
   fstream file;
   int choice;

   do {
      cout << "\n===== Employee File Handling =====";
      cout << "\n1. Add Employee Record";
      cout << "\n2. Display Employee Records";
```

```
    cout << "\n3. Exit";
    cout << "\nEnter your choice: ";
    cin >> choice;

    switch (choice) {

    case 1:
        // Write record to file
        file.open("employee.dat", ios::out | ios::app | ios::binary);

        emp.getData();

        file.write((char*)&emp, sizeof(emp));

        file.close();

        cout << "\nRecord Stored Successfully!";
        break;

    case 2:
        // Read records from file
        file.open("employee.dat", ios::in | ios::binary);

        if (!file) {
            cout << "\nFile not found!";
            break;
        }

        cout << "\n--- Employee Records ---";

        while (file.read((char*)&emp, sizeof(emp))) {
            emp.showData();
        }

        file.close();
        break;

    case 3:
        cout << "\nExiting Program...";
        break;

    default:
        cout << "\nInvalid Choice!";
    }

} while (choice != 3);
```

```
    return 0;
}
```

Practical 7 : CG : CG01.cpp (DDA line and Bresenham's Circle)

Write a C++ Program to draw the following pattern using The
DDA line and Bresenham's Circle drawing algorithm.

```
#include<iostream>
#include<graphics.h>
#include<math.h>
using namespace std;
class line
{

  int x,y,dx,dy,xi,yi,steps;
  public:

void put()
{
putpixel(x,y,9);
}

 void lin(int x1,int y1,int x2, int y2)
{

dx=x2-x1;
dy=y2-y1;
if(abs(dx)>abs(dy))
  steps=abs(dx);
else
steps=abs(dy);

xi=dx/steps;
yi=dy/steps;

x=x1+0.5;
y=y1+0.5;
put();

for (int  i=1;i<=steps;i++)
{
x=x+xi;
y=y+yi;
put();
delay(5);
}
}
}l;
```

```
class circle1
{
public :

void circle(int x,int y,int rad)
{
int x1=0;
int y1=rad;
putpixel(x1,y1,6);
int pk=3-2*rad;
while(x1<y1)
{
if(pk<=0)
{
pk=pk+(4*x1)+6;
}
else
{
pk=pk+4*(x1-y1)+10;
y1--;
}
x1++;
putpixel(x+x1,y+y1,3);
putpixel(x-x1,y+y1,3);
putpixel(x+x1,y-y1,3);
putpixel(x-x1,y-y1,3);
putpixel(x+y1,y+x1,3);
putpixel(x-y1,y+x1,3);
putpixel(x+y1,y-x1,3);
putpixel(x-y1,y-x1,3);
delay(10);
}
}
}c;
int main()
{
 int gd=DETECT,gm=DETECT;
 initgraph(&gd,&gm,NULL);
 l.lin(100,100,400,100);
 l.lin(100,400,400,400);
 l.lin(100,100,100,400);
 l.lin(400,100,400,400);
 l.lin(250,100,100,250);
 l.lin(100,250,250,400);
 l.lin(250,100,400,250);
 l.lin(400,250,250,400);
 /*l.lin(175,175,325,175);
```

```
 l.lin(175,325,325,325);
 l.lin(175,175,175,325);
 l.lin(325,175,325,325);*/
 c.circle(250,250,100);
 delay(500);
closegraph();
return 0;
}
```

Practical 8: CG CG02.cpp (Cohen–Sutherland line clipping algorithm)

Write a C++ program to implement the Cohen–SuCG therland line clipping algorithm for clipping a line against a rectangular clipping window.

```
#include<iostream>
#include<graphics.h>
#include<math.h>
using namespace std;

static int LEFT=1, RIGHT=2, BOTTOM=4, TOP=8,xl,yl,xh,yh;

int getcode(int x,int y)
{
 int code=0;
 if(y>yh) code |=TOP;
 if(y<yl) code |=BOTTOM;
 if(x<xl) code |=LEFT;
 if(x>xh) code |=RIGHT;
 return code;
}

int main()
{
 int gdriver=DETECT,gmode;
 int x1,y1,x2,y2;
 cout<<"Enter Bottom-Left And Top-Right Coordinates Of Windows:";
 cin>>xl>>yl>>xh>>yh;
 cout<<"Enter The ENd-Point Coordinates Of Line:";
 cin>>x1>>y1>>x2>>y2;
 initgraph(&gdriver,&gmode,NULL);
 setcolor(BLUE);
 rectangle(xl,yl,xh,yh);
 line(x1,y1,x2,y2);
 getch();

 int outcode1=getcode(x1,y1), outcode2=getcode(x2,y2);
 int accept=0;

while(1)
{
 float m=(float)(y2-y1)/(x2-x1);
 if (outcode1==0 && outcode2==0)
   {
     accept=1;
     break;
   }
 else if((outcode1 & outcode2)!=0)
   {
     break;
```

```
  }
 else
  {
  int x,y;
  int temp;

  if(outcode1==0)
  {
   temp=outcode2;
  }
  else
  {
   temp=outcode1;
  }

  if(temp & TOP)
  {
   x=x1+(yh-y1)/m;
   y=yh;
  }
  else if(temp & BOTTOM)
  {
   x=x1+(yl-y1)/m;
   y=yl;
  }
  else if(temp & LEFT)
  {
   x=xl;
   y=y1+m*(xl-x1);
  }
  else if(temp & RIGHT)
  {
   x=xh;
   y=y1+m*(xh-x1);
  }


  if(temp==outcode1)
  {
   x1=x;
   y1=y;
   outcode1=getcode(x1,y1);
  }
  else
  {
   x2=x;
   y2=y;
   outcode2=getcode(x2,y2);
  }
 }
}
```

```
cout<<"After Clipping :";
if(accept)
  cleardevice();
  rectangle(xl,yl,xh,yh);
  setcolor(WHITE);
  line(x1,y1,x2,y2);
  delay(5000);
  closegraph();
getch();
return 0;
}
```

Practical 9 : CG CG03.cpp (2-D transformations such as translation, scaling, rotation, reflection, and shearing)

Write a C++ program to implement 2-D transformations such as translation, scaling, rotation, reflection, and shearing on a given object.

```
#include<iostream>
#include<graphics.h>

using namespace std;
class drawpoly
{
public:
int x,y, temp,n,i,j,k,dy,dx;
int a[20][2],xi[20];
float slope[20];

void drawpoly1()
{

cout<<"\n enter the no .of vertices of polygoan";
cin>>n;
cout<<"\n enter the coordinates of edges one by one";
for(i=0;i<n;i++)
{
cout<<"X"<<i<<"Y"<<i<<":";
  cin>>a[i][0]>>a[i][1];

}
a[n][0]=a[0][0];//last point should be connceted to first point to make a close fig
a[n][1]=a[0][1];

for(i=0;i<n;i++)
{
line(a[i][0],a[i][1],a[i+1][0],a[i+1][1]);// draw poly
   }
getch();

  }

}d;
class fillpoly : public drawpoly
{
 public:
  void fillpoly1()
  {

for(i=0;i<n;i++)
{
```

```
dy=a[i+1][1]-a[i][1];  //dy=y2-y1
dx=a[i+1][0]-a[i][0];   //dx=x2-x1
if(dy==0)
 slope[i]=1.0; // y is always one because scanline diff is only by -1
if(dx==0)
 slope[i]=0.0;
if(dy!=0 && dx!=0)  //calculate inverse slope if polygon is in 4th quadradent.
{
slope[i]=(float)dx/dy; //formula for inverse slope
}
}

for(y=0;y<480;y++) // for reverse filling(y=480;y>0;y--)
{
k=0;
for(i=0;i<n;i++)
{
if(((a[i][1]<=y) && (a[i+1][1]>y))||((a[i][1]>y)&& (a[i+1][1]<=y) ))// check ymin or ymax is less or greater
{
xi[k]=(int)(a[i][0])+slope[i]*(y-a[i][1]);// calculate intersection
k++;
}
}
for(j=0;j<k-1;j++)
{
for(i=0;i<k-1;i++)
{
if(xi[i]>xi[i+1])
{
temp=xi[i];
xi[i]=xi[i+1];
xi[i+1]=temp;

}
}
setcolor(13);
for(i=0;i<k;i+=2)
{
line(xi[i],y,xi[i+1],y);//draw the line using intersection points only
delay(20);
}
}
}
}
}f;
int main()
{
int gd=DETECT,gm;
initgraph(&gd,&gm,NULL);
  f.drawpoly1();
     f.fillpoly1();
```

```
closegraph();
return 0;
}
```

Practical 10 : CG CG04.cpp (concave polygon filling)

A concave polygon filling using scan fill algorithm

```
#include<iostream>
#include<math.h>
#include<graphics.h>
using namespace std;
class matrix
{
public:
   int n,i,j,tx,ty,k,sum,sx,sy;
   double a[6][3],b[6][3],mult[6][3],mat3[6][3];
   double p,q,r;
   double ang=0,angle=0;
public:
   void get()
     {
   cout<<"\n enter the number of vertices of polygon : ";
     cin>>n;
    // cout<<"n Entering user matix\n";
     for(i=0;i<n;i++)
     {
      cout<<"enter x n y co ordinates";
        cin>>b[i][0];
        cin>>b[i][1];
        b[i][2]=1;
     }
//display object matrix
     cout<<"\n original  co ordinates are"<<"\n";

     for(i=0;i<n;i++)
     {
        for(j=0;j<3;j++)
        {
           cout<<b[i][j]<<"\t";
        }cout<<"\n";
     }
     }
void identitymat()
{
for(i=0;i<n;i++)
      {
            for(j=0;j<3;j++)
            {
               if(i==j)
               {
                  a[i][j]=1;
               }
               else
               {
```

```
                a[i][j]=0;
              }
            }
        }
}
   void trans()
   {

      cout<<"enter values of tx and ty";
      cin>>tx>>ty;
      a[2][0]=tx;
      a[2][1]=ty;
      cout<<"matrix is"<<"\n";
      for(i=0;i<n;i++)
      {
         for(j=0;j<3;j++)
         {
            cout<<a[i][j]<<"\t";
         }cout<<"\n";
      }
    }
   void scale()
   {
      cout<<"\n Enter the values of sx and sy";
      cin>>sx>>sy;
      a[0][0]=sx;
      a[1][1]=sy;
   cout<<"\n Matrix is:"<<"\n";
//To display scaling matrix
      for(i=0;i<3;i++)
      {
            for(j=0;j<3;j++)
      {
               cout<<a[i][j]<<"\t";
      }cout<<"\n";
      }
   }
   void rot()
     {
      cout<<"Enter the angle";
     cin>>ang;
     angle=(ang*3.142)/180;
     q=sin(angle);
     p=cos(angle);
     r=-sin(angle);
     a[0][0]=p;
     a[0][1]=q;
     a[1][0]=r;
     a[1][1]=p;
     cout<<"tranformation matrix is"<<"\n";
```

```
    for(i=0;i<3;i++)
    {
       for(j=0;j<3;j++)
       {
          cout<<a[i][j]<<"\t";
       }cout<<"\n";
    }
    }
  void multi()
       {
          cout<<"\nMultiplying two matrices...";
             for(i=0; i<n; i++)
             {
                for(j=0; j<3; j++)
                {
                   sum=0;
                   for(k=0; k<3; k++)
                   {
                      sum = sum + b[i][k] * a[k][j];
                   }
                   mat3[i][j] = sum;
                }
             }
       }
       void display()
       {
             cout<<"\nMultiplication of two Matrices : \n";
             for(i=0; i<n; i++)
             {
                for(j=0; j<3; j++)
                {
                   cout<<mat3[i][j]<<" ";
                }
                cout<<"\n";
             }

int gd=DETECT,gm;
initgraph(&gd,&gm,NULL);

for(int i=0;i<n-1;i++)
{
   line(b[i][0],b[i][1],b[i+1][0],b[i+1][1]);
}
line(b[2][0],b[2][1],b[0][0],b[0][1]);
for(int i=0;i<n-1;i++)
{
   line(mat3[i][0],mat3[i][1],mat3[i+1][0],mat3[i+1][1]);
}
line(mat3[2][0],mat3[2][1],mat3[0][0],mat3[0][1]);
delay(5000);
closegraph();
```

```
        }
    };
int main()
{
    matrix g;
    int ch;
    char ans;
    g.get();
    g.identitymat();
    do
    {
cout<<"menu\n1.translation\n2.scaling\n3.rotation";
    cin>>ch;
    switch(ch)
    {
    case 1:
        g.trans();
        g.multi();
        g.display();
        break;
    case 2:
        g.scale();
        g.multi();
        g.display();
        break;
    case 3:
        g.rot();
        g.multi();
        g.display();
        break;
    }cin>>ans;
    }while(ans=='Y'&& ans=='y');
return 0;
}
```

All CG Codes Given By VM :

Assignment 1 part 1 :

```
#include<iostream>
#include<graphics.h>
#include<math.h>
using namespace std;
class line
{

  int x,y,dx,dy,xi,yi,steps;
  public:

void put()
{
putpixel(x,y,9);
}

 void lin(int x1,int y1,int x2, int y2)
{

dx=x2-x1;
dy=y2-y1;
if(abs(dx)>abs(dy))
  steps=abs(dx);
else
steps=abs(dy);

xi=dx/steps;
yi=dy/steps;

x=x1+0.5;
y=y1+0.5;
put();

for (int  i=1;i<=steps;i++)
{
x=x+xi;
y=y+yi;
put();
delay(5);
}
}
}l;

class circle1
{
public :

void circle(int x,int y,int rad)
```

```
{
int x1=0;
int y1=rad;
putpixel(x1,y1,6);
int pk=3-2*rad;
while(x1<y1)
{
if(pk<=0)
{
pk=pk+(4*x1)+6;
}
else
{
pk=pk+4*(x1-y1)+10;
y1--;
}
x1++;
putpixel(x+x1,y+y1,3);
putpixel(x-x1,y+y1,3);
putpixel(x+x1,y-y1,3);
putpixel(x-x1,y-y1,3);
putpixel(x+y1,y+x1,3);
putpixel(x-y1,y+x1,3);
putpixel(x+y1,y-x1,3);
putpixel(x-y1,y-x1,3);
delay(10);
}
}
}c;
int main()
{
 int gd=DETECT,gm=DETECT;
 initgraph(&gd,&gm,NULL);
 l.lin(100,100,400,100);
 l.lin(100,400,400,400);
 l.lin(100,100,100,400);
 l.lin(400,100,400,400);
 l.lin(250,100,100,250);
 l.lin(100,250,250,400);
 l.lin(250,100,400,250);
 l.lin(400,250,250,400);
 /*l.lin(175,175,325,175);
 l.lin(175,325,325,325);
 l.lin(175,175,175,325);
 l.lin(325,175,325,325);*/
 c.circle(250,250,100);
 delay(500);
closegraph();
return 0;
}
```

Assignment 1 part 2 :

```
#include<iostream>
#include<graphics.h>
#include<math.h>
using namespace std;

class line
{
  public:
  int x,y,dx,dy,m,steps,xi,yi;
  void output()
    {
      putpixel(x,y,10);
    }
  void line1(int x1,int y1,int x2, int y2)
    {
      dx=x2-x1;
      dy=y2-y1;
      if(abs(dx)>abs(dy))
        steps=abs(dx);
      else
        steps=abs(dy);

      xi=dx/steps;
      yi=dy/steps;
      x=x1+0.5;
      y=y1+0.5;
      output();

      for(int i=1;i<=steps;i++)
        {
          x=x+xi;
          y=y+yi;
          output();
          delay(10);
        }
    }
}l;
class circle1
{
 public:
  void drawcircle(int x,int y,int r)
{
  int x1=0,y1=r;
  putpixel(x1,y1,3);
  int dk=3-2*r;
  while(x1<y1)
  {
    if(dk<=0)
    {
      dk=dk+(4*x1)+6;
```

```
    }
    else
    {
      dk=dk+(4*(x1-y1))+10;
      y1--;
    }
    x1++;
  putpixel(x+x1,y+y1,3);
  putpixel(x-x1,y+y1,3);
  putpixel(x+x1,y-y1,3);
  putpixel(x-x1,y-y1,3);
  putpixel(x+y1,y+x1,3);
  putpixel(x-y1,y+x1,3);
  putpixel(x+y1,y-x1,3);
  putpixel(x-y1,y-x1,3);
  delay(15);
  }
}
}c;

int main()
{
  int gd=DETECT,gm=DETECT;
  initgraph(&gd,&gm,NULL);
  l.line1(250,250,150,150);
  l.line1(150,150,50,250);
  l.line1(50,250,250,250);
/* l.line1(100,100,100,400);
 l.line1(100,250,250,400);
l.line1(250,400,400,250);
  l.line1(400,250,250,100);
 l.line1(250,100,100,250);*/
  //l.line1(175,325,325,325);
  //l.line1(325,325,325,175);
 // l.line1(325,175,175,175);
 // l.line1(175,175,175,325);
  c.drawcircle(149,249,100);
  c.drawcircle(150,210,40);
  delay(500);
  //r=95;x=75;y=95;
  getch();
  closegraph();
  return 0;
}
```

Assignment 2 :

```
#include<iostream>
#include<graphics.h>
using namespace std;

static int LEFT=1,RIGHT=2,BOTTOM=4,TOP=8,xl,yl,xh,yh;

int getcode(int x,int y){
int code = 0;
//Perform Bitwise OR to get outcode
if(y > yh) code |=TOP;
if(y < yl) code |=BOTTOM;
if(x < xl) code |=LEFT;
if(x > xh) code |=RIGHT;
return code;
}

int main()
{
int gdriver = DETECT,gmode;

cout<<"Enter bottom left and top right co-ordinates of window: ";
cin>>xl>>yl>>xh>>yh;

int x1,y1,x2,y2;
cout<<"Enter the endpoints of the line: ";
cin>>x1>>y1>>x2>>y2;
initgraph(&gdriver,&gmode,NULL);
setcolor(BLUE);
rectangle(xl,yl,xh,yh);
line(x1,y1,x2,y2);
getch();

int outcode1=getcode(x1,y1), outcode2=getcode(x2,y2);
int accept = 0; //decides if line is to be drawn
while(1){
float m =(float)(y2-y1)/(x2-x1);
//Both points inside. Accept line
if(outcode1==0 && outcode2==0){
accept = 1;
break;
}
//AND of both codes != 0.Line is outside. Reject line
else if((outcode1 & outcode2)!=0){
break;
}else{
int x,y;
int temp;
//Decide if point1 is inside, if not, calculate intersection
if(outcode1==0)
temp = outcode2;
```

```
else
temp = outcode1;
//Line clips top edge
if(temp & TOP){
x = x1+ (yh-y1)/m;
y = yh;
}
else if(temp & BOTTOM){ //Line clips bottom edge
x = x1+ (yl-y1)/m;
y = yl;
}else if(temp & LEFT){ //Line clips left edge
x = xl;
y = y1+ m*(xl-x1);
}else if(temp & RIGHT){ //Line clips right edge
x = xh;
y = y1+ m*(xh-x1);
}
//Check which point we had selected earlier as temp, and replace its co-ordinates
if(temp == outcode1){
x1 = x;
y1 = y;
outcode1 = getcode(x1,y1);
}else{
x2 = x;
y2 = y;
outcode2 = getcode(x2,y2);
}
}
}

cout<<"After clipping:";
if(accept)
cleardevice();
rectangle(xl,yl,xh,yh);
setcolor(WHITE);
line(x1,y1,x2,y2);
delay(5000);
closegraph();
return 0;
}
```

Assignment 3 :

```
#include<iostream>
#include<graphics.h>

using namespace std;
class drawpoly
{
public:
int x,y, temp,n,i,j,k,dy,dx;
int a[20][2],xi[20];
float slope[20];

void drawpoly1()
{

cout<<"\n enter the no .of vertices of polygoan";
cin>>n;
cout<<"\n enter the coordinates of edges one by one";
for(i=0;i<n;i++)
{
cout<<"X"<<i<<"Y"<<i<<":";
  cin>>a[i][0]>>a[i][1];

}
a[n][0]=a[0][0];//last point should be connceted to first point to make a close fig
a[n][1]=a[0][1];

for(i=0;i<n;i++)
{
line(a[i][0],a[i][1],a[i+1][0],a[i+1][1]);// draw poly
   }
getch();

  }

}d;
class fillpoly : public drawpoly
{
 public:
  void fillpoly1()
  {

for(i=0;i<n;i++)
{
dy=a[i+1][1]-a[i][1];  //dy=y2-y1
dx=a[i+1][0]-a[i][0];   //dx=x2-x1
if(dy==0)
 slope[i]=1.0; // y is always one because scanline diff is only by -1
if(dx==0)
 slope[i]=0.0;
```

```
if(dy!=0 && dx!=0)  //calculate inverse slope if polygon is in 4th quadradent.
{
slope[i]=(float)dx/dy; //formula for inverse slope
}
}

for(y=0;y<480;y++) // for reverse filling(y=480;y>0;y--)
{
k=0;
for(i=0;i<n;i++)
{
if(((a[i][1]<=y) && (a[i+1][1]>y))||((a[i][1]>y)&& (a[i+1][1]<=y) ))// check ymin or ymax is less or greater
{
xi[k]=(int)(a[i][0])+slope[i]*(y-a[i][1]);// calculate intersection
k++;
}
}
for(j=0;j<k-1;j++)
{
for(i=0;i<k-1;i++)
{
if(xi[i]>xi[i+1])
{
temp=xi[i];
xi[i]=xi[i+1];
xi[i+1]=temp;

}
}
setcolor(13);
for(i=0;i<k;i+=2)
{
line(xi[i],y,xi[i+1],y);//draw the line using intersection points only
delay(20);
}
}
}
}
}f;
int main()
{
int gd=DETECT,gm;
initgraph(&gd,&gm,NULL);
  f.drawpoly1();
     f.fillpoly1();
closegraph();
return 0;
}
```

Assignment 4 :

```
#include<iostream>
#include<math.h>
#include<graphics.h>
using namespace std;
class matrix
{
public:
   int n,i,j,tx,ty,k,sum,sx,sy;
   double a[6][3],b[6][3],mult[6][3],mat3[6][3];
   double p,q,r;
   double ang=0,angle=0;
public:
   void get()
     {
   cout<<"\n enter the number of vertices of polygon : ";
     cin>>n;
    // cout<<"n Entering user matix\n";
     for(i=0;i<n;i++)
     {
      cout<<"enter x n y co ordinates";
        cin>>b[i][0];
        cin>>b[i][1];
        b[i][2]=1;
     }
//display object matrix
     cout<<"\n original  co ordinates are"<<"\n";

     for(i=0;i<n;i++)
     {
        for(j=0;j<3;j++)
        {
           cout<<b[i][j]<<"\t";
        }cout<<"\n";
     }
     }
void identitymat()
{
for(i=0;i<n;i++)
      {
            for(j=0;j<3;j++)
            {
               if(i==j)
               {
                  a[i][j]=1;
               }
               else
               {
                  a[i][j]=0;
               }
            }
```

```
    }
}
  void trans()
  {

    cout<<"enter values of tx and ty";
    cin>>tx>>ty;
    a[2][0]=tx;
    a[2][1]=ty;
    cout<<"matrix is"<<"\n";
    for(i=0;i<n;i++)
    {
       for(j=0;j<3;j++)
       {
          cout<<a[i][j]<<"\t";
       }cout<<"\n";
    }
   }
  void scale()
  {
    cout<<"\n Enter the values of sx and sy";
    cin>>sx>>sy;
    a[0][0]=sx;
    a[1][1]=sy;
  cout<<"\n Matrix is:"<<"\n";
//To display scaling matrix
    for(i=0;i<3;i++)
    {
         for(j=0;j<3;j++)
    {
              cout<<a[i][j]<<"\t";
    }cout<<"\n";
    }
  }
  void rot()
    {
     cout<<"Enter the angle";
    cin>>ang;
    angle=(ang*3.142)/180;
    q=sin(angle);
    p=cos(angle);
    r=-sin(angle);
    a[0][0]=p;
    a[0][1]=q;
    a[1][0]=r;
    a[1][1]=p;
    cout<<"tranformation matrix is"<<"\n";

    for(i=0;i<3;i++)
    {
       for(j=0;j<3;j++)
```

```
        {
           cout<<a[i][j]<<"\t";
        }cout<<"\n";
     }
     }
   void multi()
        {
           cout<<"\nMultiplying two matrices...";
              for(i=0; i<n; i++)
              {
                 for(j=0; j<3; j++)
                 {
                    sum=0;
                    for(k=0; k<3; k++)
                    {
                       sum = sum + b[i][k] * a[k][j];
                    }
                    mat3[i][j] = sum;
                 }
              }
        }
        void display()
        {
              cout<<"\nMultiplication of two Matrices : \n";
              for(i=0; i<n; i++)
              {
                 for(j=0; j<3; j++)
                 {
                    cout<<mat3[i][j]<<" ";
                 }
                 cout<<"\n";
              }

int gd=DETECT,gm;
initgraph(&gd,&gm,NULL);

for(int i=0;i<n-1;i++)
{
   line(b[i][0],b[i][1],b[i+1][0],b[i+1][1]);
}
line(b[2][0],b[2][1],b[0][0],b[0][1]);
for(int i=0;i<n-1;i++)
{
   line(mat3[i][0],mat3[i][1],mat3[i+1][0],mat3[i+1][1]);
}
line(mat3[2][0],mat3[2][1],mat3[0][0],mat3[0][1]);
delay(5000);
closegraph();
        }
   };
int main()
```

```
{
   matrix g;
   int ch;
   char ans;
   g.get();
   g.identitymat();
   do
   {
cout<<"menu\n1.translation\n2.scaling\n3.rotation";
   cin>>ch;
   switch(ch)
   {
   case 1:
      g.trans();
      g.multi();
      g.display();
      break;
   case 2:
      g.scale();
      g.multi();
      g.display();
      break;
   case 3:
      g.rot();
      g.multi();
      g.display();
      break;
   }cin>>ans;
   }while(ans=='Y'&& ans=='y');
return 0;
}
```